

P2893R2

Variadic friends

Published Proposal, 2024-02-09

Authors:

[Jody Hagins](#)

[Arthur O'Dwyer](#)

Audience:

CWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Support `friend Ts...` in C++26.

Table of Contents

1	Changelog
2	Introduction
2.1	Passkey idiom
2.2	CRTTP access to derived
3	Confusing grammatical examples
3.1	<code>template<class... Ts> friend Ts...</code>
3.2	<code>friend class Ts...</code>
4	Proposed wording
4.1	Option 1 (preferred)
4.1.1	[cpp.predefined]
4.1.2	[class.mem.general]
4.1.3	[dcl.pre]
4.1.4	[temp.pre]
4.1.5	[class.friend]
4.1.6	[dcl.type.elab]
4.1.7	[temp.res.general]
4.1.8	[temp.variadic]
4.2	Option 2
4.2.1	[cpp.predefined]
4.2.2	[dcl.type.elab]
4.2.3	[dcl.type.general]
4.2.4	[class.friend]

5 Acknowledgments

§ 1. Changelog

- R2 (post-Kona 2023):
 - New and improved wording after CWG feedback ([minutes](#))
 - Add feature-test macro `__cpp_variadic_friend`

§ 2. Introduction

This paper proposes support for granting friendship to all classes in a parameter pack. Several existing idioms are implemented by providing friendship to a class via template parameter. However, these patterns can only be used with a single template parameter, because friendship cannot be currently granted to a pack of types.

Before	After
<pre>template<class T=void, class U=void> class Foo { friend T; friend U; };</pre>	<pre>template<class... Ts> class Foo { friend Ts...; };</pre>

§ 2.1. Passkey idiom

The Passkey idiom allows granting access to individual member functions on a per-function basis. In the example below, `C` grants friendship to `A`, meaning that `A::m` can access all of `C`'s internals. But `C` also grants access to `intentionalA` and `intentionalB` using the Passkey idiom. Class `Passkey<B>` has a private constructor accessible only to its friend `B`, so nobody but `B` can construct instances of `Passkey<B>`. You can't call `intentionalB` without an instance of `Passkey<B>` as the first argument. So, even though `intentionalB` is public, it is callable only from `B`.

```
template<class T>
class Passkey {
    friend T;
    Passkey() {}
};

class A;
class B;

class C {
    friend A;
private:
    void internal();
public:
    void intentionalA(Passkey<A>);
    void intentionalB(Passkey<B>);
};
```

```

class A {
    void m(C& c) {
        c.internal(); // OK
        c.intentionalA({}); // OK
        c.intentionalB({}); // Error, Passkey<B>'s ctor is inaccessible
    }
};

class B {
    void m(C& c) {
        c.intentionalB({}); // OK
    }
};

```

We would like to expand this idiom, and grant access to `Foo::intentional` from multiple classes.

```

template<class... Ts>
class Passkey {
    friend Ts...; // Today: Error. Proposed: OK
    Passkey() {}
};

class C {
public:
    // Only callable from Blarg, Blip, and Baz
    void intentional(Passkey<Blarg, Blip, Baz>);
};

```

§ 2.2. CRTP access to derived

Another common pattern is to inherit from some class template, passing the type of the derived class as a template parameter to the base class.

There may be parts of the derived class API which are needed in the base class, but only the base class, so they are private, and friendship is granted to the base class.

```

template<class Crtp, class MsgT>
class Receiver {
    void receive(MsgT) {
        static_cast<Crtp*>(this)->private_ += 1;
    }
};

template<class MsgT>
struct Dispatcher :
    public Receiver<Dispatcher<MsgT>, MsgT>
{
    using Receiver<Dispatcher, MsgT>::Receiver;
    friend Receiver<Dispatcher, MsgT>;

private:
    int private_;
};

```

To support multiple base classes, we would like to make `Dispatcher` variadic:

```
template<class Crtp, class MsgT>
class Receiver {
    void receive(MsgT) {
        static_cast<Crtp*>(this)->private_ += 1;
    }
};

template<class... MsgTs>
struct Dispatcher :
    public Receiver<Dispatcher<MsgTs...>, MsgTs>... // OK
{
    using Receiver<Dispatcher, MsgTs>::Receiver...; // OK
    friend Receiver<Dispatcher, MsgTs>...; // Today: Error. Proposed: OK

private:
    int private_;
};
```

Note that both inheritance and `using` support pack-expansion. Only `friend` does not.

§ 3. Confusing grammatical examples

The following two subsections merely record some design notes in case we want to refer back to them during CWG discussion. You can skip reading these subsections for now.

§ 3.1. `template<class... Ts> friend Ts...`

A declaration `template<class T> friend ~~~` is not a template for stamping out friends; it is a request to befriend a specific template.

```
template<class T>
friend class C::Nested;
```

declares that `C` has a member named `Nested` which itself is a class template, and we’re befriendng that template. That is:

```
struct C { template<class T> class Nested; };
struct S {
    template<class T>
    friend class C::Nested;
};
```

Therefore it is never well-formed to use the parameters of the friend declaration’s own *template-head* within the declarator itself. That is, these are all ill-formed, and replacing today’s `U` with tomorrow’s `Us...` won’t change anything:

Before	After
<pre>template<class T> struct S { template<class U> friend U; // ill-formed</pre>	<pre>template<class... Ts> struct VS { template<class... Us> friend Us...; // ill-formed</pre>

<pre> template<class U> friend class U; // ill-formed template<class U> friend class C<T>::Nested<U>; // ill-formed template<class U> friend class C<U>::Nested; // ill-formed }; </pre>	<pre> template<class... Us> friend class Us...; // ill-formed template<class U> friend class C<Ts>::Nested<U>...; // ill-formed template<class... Us> friend class C<Ts...>::Nested<Us>...; // ill-formed template<class... Us> friend class C<Us>::Nested...; // ill-formed }; </pre>
--	---

On the other hand, this usage (to befriend the member template `C<T>::Nested`) is well-formed, and remains well-formed after replacing today’s `T` with tomorrow’s `Ts...`:

Before	After
<pre> template<class T> struct C { template<class U> struct Nested; }; template<class T> struct S { template<class U> friend class C<T>::Nested; // OK }; template<class T> template<class U> struct C<T>::Nested { int m(S<int>& s) { return s.private_; } }; int main() { S<int> s; C<int>::Nested<float>().m(s); // OK C<float>::Nested<float>().m(s); // error, inaccessible } </pre>	<pre> template<class T> struct C { template<class U> struct Nested; }; template<class... Ts> struct VS { template<class U> friend class C<Ts>::Nested...; // OK }; template<class T> template<class U> struct C<T>::Nested { int m(VS<int, char>& vs) { return vs.private_; } }; int main() { VS<int, char> vs; C<int>::Nested<float>().m(vs); // OK C<char>::Nested<float>().m(vs); // OK C<float>::Nested<float>().m(vs); // error, inaccessible } </pre>

§ 3.2. friend class `Ts...`

C++ already disallows `friend class T` when it stands alone ([Godbolt](#)):

Before	After
<pre> template<class T> struct S { friend class T; // Error }; </pre>	<pre> template<class... Ts> struct S { friend class Ts...; // Error }; </pre>

but permits these ways to befriend a specific class instantiated from a class template `C`, or to befriend a member class template:

Before	After
<pre>template<class T> struct S { friend class C<T>; friend class N::C<T>; };</pre>	<pre>template<class... Ts> struct S { friend class C<Ts>...; friend class N::C<Ts>...; };</pre>
<pre>template<class T> struct S { template<class U> friend class C<T>::Nested; };</pre>	<pre>template<class... Ts> struct S { template<class U> friend class C<Ts>::Nested...; };</pre>

The existing wording for this is in [\[dcl.type.elab\]/4](#), which we modify [below](#) by adding an `opt`.

§ 4. Proposed wording

We present two possible wording options: our preferred way, and an alternative for comparison.

Option 1	Option 2
<code>friend Ts...</code> expands to <code>friend T1, T2</code>	<code>friend Ts...</code> expands to <code>friend T1; friend T2</code>
<code>friend T1, T2;</code> is OK	<code>friend T1, T2;</code> remains ill-formed
Consistent with <code>using T1::T1, T2::T2;</code>	—
Does the work up front	We expect future proposals for <code>friend T1, T2;</code> anyway
Adds <i>friend-type-declaration</i>	Adds no new nonterminals
Fails to disentangle the whole <code>friend</code> grammar	Doesn't even <i>try</i> to disentangle the <code>friend</code> grammar
Wording seems OK	Wording apparently permits <code>void f(int... xs);</code> and <code>sizeof(int...)</code>
We prefer Option 1.	Provided for comparison only.

P2893R0 presented Option 1, but with incomplete wording. At Kona, EWG was concerned that the wording for `friend T1, T2` might be difficult, and so the presentation drifted toward Option 2. But, having investigated Option 2, we think it's really about as hard, and has the drawbacks above. We show it here for comparison, so that when you ask "Why not just do [basically Option 2]?" we can point to it and say "Look. We tried. It's no better."

Relevant paragraphs not modified here include:

- [\[dcl.meaning.general\]/2](#) "If the declaration is a friend declaration, the declarator does not bind a name..."
- [\[temp.friend\]](#) "A friend template may be declared within a class or class template ... A friend template shall not be declared in a local class ..."

§ 4.1. Option 1 (preferred)

NOTE: Precedent for "If a *member-declaration* matches..." is found in [\[dcl.pre\]/10](#) "If a *static-assert-message* matches..."

NOTE: CWG discussion in Kona ([minutes](#)) suggested that *friend-type-specifier* shouldn't be producible from *elaborated-type-specifier*, because then we'd appear to support `friend class A, class B;`. However, as shown above, we need to support `template<class> friend class Ts::Nested...`; which means that *friend-type-specifier* must be producible from *elaborated-type-specifier*. Luckily [\[dcl.type.elab\]/4](#) already forbids `friend class A, class B;`. We add `...opt` to [\[dcl.type.elab\]/4](#) but don't add anything to permit commas.

NOTE: Today `friend typename X<T>::Y;` is legal, as is `friend X<T>::Y;`. The identifier there is in a type-only context ([\[temp.res.general\]](#)) because it's a *decl-specifier* of the *decl-specifier-seq* of a *member-declaration*. After our patch, it's in a type-only context because it's the *friend-type-specifier* of a *friend-type-declaration*.

NOTE: To befriend a non-template, you use a "friend declaration" which is a *member-declaration*. To befriend a template, you use a *template-declaration*, which consists of a *template-head* followed by a *declaration*; the "friend declaration" here is the *declaration* ([\[class.friend\]/3](#)). Therefore, *friend-type-declaration* must appear in the list of alternatives for both *member-declaration* and *declaration*. Friend declarations are already forbidden to appear outside of classes, although that wording is elusive (maybe it doesn't exist).

§ 4.1.1. [cpp.predefined]

Add a feature-test macro to the table in [\[cpp.predefined\]](#):

```
__cpp_variable_templates 201304L
__cpp_variadic_friend     YYYYMM
__cpp_variadic_templates 200704L
__cpp_variadic_using      201611L
```

§ 4.1.2. [class.mem.general]

Modify [\[class.mem.general\]](#) as follows:

member-declaration:

*attribute-specifier-seq*_{opt} *decl-specifier-seq*_{opt} *member-declarator-list*_{opt} ;

function-definition

friend-type-declaration

using-declaration

using-enum-declaration

static_assert-declaration

template-declaration

explicit-specialization

deduction-guide

alias-declaration

opaque-enum-declaration

empty-declaration

friend-type-declaration:

friend *friend-type-specifier-list* ;

friend-type-specifier-list:

friend-type-specifier ..._{opt}

friend-type-specifier-list , *friend-type-specifier* ..._{opt}

friend-type-specifier:

simple-type-specifier

elaborated-type-specifier

typename-specifier

member-declarator-list:

member-declarator

member-declarator-list , *member-declarator*

[...]

2. A *member-declaration* does not declare new members of the class if it is

- a *friend declaration* ([*class.friend*]),
- a *deduction-guide* ([*temp.deduct-guide*]),
- a *template-declaration* whose declaration is one of the above,
- a *static_assert-declaration*,
- a *using-declaration* ([*namespace.udecl*]), or
- an *empty-declaration*.

For any other *member-declaration*, each declared entity that is not an unnamed bit-field is a member of the class, and each such *member-declaration* shall either declare at least one member name of the class or declare at least one unnamed bit-field.

[...]

8. A class *C* is complete at a program point *P* if the definition of *C* is reachable from *P* ([*module.reach*]) or if *P* is in a complete-class context of *C*. Otherwise, *C* is incomplete at *P*.

x. If a *member-declaration* matches the syntactic requirements of *friend-type-declaration*, it is a *friend-type-declaration*.

9. In a *member-declarator*, an = immediately following the *declarator* is interpreted as introducing a *pure-specifier* if the *declarator-id* has function type; otherwise it is interpreted as introducing a *brace-or-equal-initializer*. [...]

10. In a *member-declarator* for a bit-field, the *constant-expression* is parsed as the longest sequence of tokens that could syntactically form a *constant-expression*. [...]

§ 4.1.3. [dcl.pre]

Modify [dcl.pre] as follows:

1. Declarations generally specify how names are to be interpreted. Declarations have the form

declaration-seq:

declaration
declaration-seq declaration

declaration:

name-declaration
special-declaration

name-declaration:

block-declaration
nodeclspec-function-declaration
function-definition
friend-type-declaration
template-declaration
deduction-guide
linkage-specification
namespace-definition
empty-declaration
attribute-declaration
module-import-declaration

special-declaration:

explicit-instantiation
explicit-specialization
export-declaration

[...]

2. Certain declarations contain one or more scopes ([*basic.scope.scope*]). Unless otherwise stated, utterances in [*dcl.dcl*] about components in, of, or contained by a declaration or subcomponent thereof refer only to those components of the declaration that are not nested within scopes nested within the declaration.

x. If a *declaration* matches the syntactic requirements of *friend-type-declaration*, it is a *friend-type-declaration*.

3. A *simple-declaration* or *nodeclspec-function-declaration* of the form [...]

§ 4.1.4. [temp.pre]

Modify [temp.pre] as follows:

1. A *template* defines a family of classes, functions, or variables, an alias for a family of types, or a concept.

template-declaration:

template-head declaration
template-head concept-definition

[...]

2. The *declaration* in a *template-declaration* (if any) shall

- declare or define a function, a class, or a variable, or
- define a member function, a member class, a member enumeration, or a static data member of a class template or of a class nested within a class template, or
- define a member template of a class or class template, or
- be a *friend-type-declaration*, or
- be a *deduction-guide*, or
- be an *alias-declaration*.

§ 4.1.5. [class.friend]

Modify [class.friend] as follows:

3. A friend declaration that does not declare a function shall be a friend-type-declaration; ~~have one of the following forms:~~

~~friend elaborated-type-specifier +~~

~~friend simple-type-specifier +~~

~~friend typename-specifier +~~

[Note: A friend declaration can be the declaration in a template-declaration ([temp.pre], [temp.friend]). — end note]

~~If the type-specifier in a friend declaration designates a (possibly cv-qualified) class type, that class is declared as a friend; otherwise, the friend declaration is ignored.~~ Each friend-type-specifier in a friend-type-declaration that designates a (possibly cv-qualified) class type declares that class as a friend. Each other friend-type-specifier is ignored.

[Example 4:

```
class C;
typedef C Ct;
struct E;

class X1 {
    friend C;                // OK, class C is a friend
};

class X2 {
    friend Ct;               // OK, class C is a friend
    friend D;               // error: D not found
    friend class D;         // OK, elaborated-type-specifier declares new class
};

template <class... Ts> class R {
    friend Ts...;
};

template <class... Ts, class... Us>
class R<R<Ts...>, R<Us...>> {
    friend Ts::Nested..., Us...;
};

R<C> rc;                    // class C is a friend of R<C>
R<C, E> rce;               // classes C and E are friends of R<C, E>
R<int> Ri;                  // OK, "friend int;" is ignored

struct E { struct Nested; };

R<R<E>, R<C, int>>> rr;     // E::Nested and C are friends of R<R<E>, R<C, int>>>
```

— end example]

§ 4.1.6. [dcl.type.elab]

Modify [dcl.type.elab] as follows:

4. ~~If an elaborated-type-specifier appears with the friend specifier as an entire member-declaration, the member-declaration shall~~ **If any friend-type-specifier in a friend-type-declaration is an elaborated-type-specifier, the friend-type-declaration shall** have one of the following forms:

- `friend class-key nested-name-specifieropt identifier ...opt ;`
- `friend class-key simple-template-id ...opt ;`
- `friend class-key nested-name-specifier templateopt simple-template-id ...opt ;`

Any unqualified lookup for the *identifier* (in the first case) does not consider scopes that contain the target scope; no name is bound.

[Note: A using-directive in the target scope is ignored if it refers to a namespace not contained by that scope. [basic.lookup.elab] describes how name lookup proceeds in an elaborated-type-specifier. — end note]

[Note: An *elaborated-type-specifier* can be used to refer to a previously declared *class-name* or *enum-name* even if the name has been hidden by a non-type declaration. — end note]

5. If the *identifier* or *simple-template-id* resolves to a *class-name* or *enum-name*, the *elaborated-type-specifier* introduces it into the declaration the same way a *simple-type-specifier* introduces its *type-name* ([dcl.type.simple]). If the *identifier* or *simple-template-id* resolves to a *typedef-name* ([dcl.typedef], [temp.names]), the *elaborated-type-specifier* is ill-formed.

[Note: This implies that, within a class template with a template *type-parameter* *T*, the declaration `friend class T;` is ill-formed. However, the similar declaration `friend T;` is well-formed ([class.friend]). — end note]

§ 4.1.7. [temp.res.general]

Modify [temp.res.general] as follows:

4. A qualified or unqualified name is said to be in a *type-only context* if it is the terminal name of

- a *typename-specifier*, *nested-name-specifier*, *elaborated-type-specifier*, *class-or-decltype*, or **the simple-type-specifier of a friend-type-specifier, or**
- a *type-specifier* of a
 - *new-type-id*,
 - *defining-type-id*,
 - *conversion-type-id*,
 - *trailing-return-type*,
 - default argument of a *type-parameter*, or
 - *type-id* of a `static_cast`, `const_cast`, `reinterpret_cast`, or `dynamic_cast`, or
- a *decl-specifier* of the *decl-specifier-seq* of a
 - *simple-declaration* or *function-definition* in namespace scope,
 - *member-declaration*,
 - *parameter-declaration* in a *member-declaration*, unless that *parameter-declaration* appears in a default argument,
 - *parameter-declaration* in a declarator of a function or function template declaration whose *declarator-id* is qualified, unless that *parameter-declaration* appears in a default argument,
 - *parameter-declaration* in a *lambda-declarator* or *requirement-parameter-list*, unless that *parameter-declaration* appears in a default argument, or
 - *parameter-declaration* of a (non-type) *template-parameter*.

§ 4.1.8. [temp.variadic]

Modify [temp.variadic] as follows:

5. A *pack expansion* consists of a *pattern* and an *ellipsis*, the instantiation of which produces zero or more instantiations of the pattern in a list (described below). The form of the pattern depends on the context in which the expansion occurs. Pack expansions can occur in the following contexts:

- In a function parameter pack ([`dcl.fct`]); the pattern is the *parameter-declaration* without the ellipsis.
- In a *using-declaration* ([`namespace.udecl`]); the pattern is a *using-declarator*.
- In a *friend-type-declaration* ([`class.mem.general`]); the pattern is a *friend-type-specifier*
- In a template parameter pack that is a pack expansion ([`temp.param`]):
 - if the template parameter pack is a *parameter-declaration*; the pattern is the *parameter-declaration* without the ellipsis;
 - if the template parameter pack is a *type-parameter*; the pattern is the corresponding *type-parameter* without the ellipsis.
- In an *initializer-list* ([`dcl.init`]); the pattern is an *initializer-clause*.
- In a *base-specifier-list* ([`class.derived`]); the pattern is a *base-specifier*.
- In a *mem-initializer-list* ([`class.base.init`]) for a *mem-initializer* whose *mem-initializer-id* denotes a base class; the pattern is the *mem-initializer*.
- In a *template-argument-list* ([`temp.arg`]); the pattern is a *template-argument*.
- In an *attribute-list* ([`dcl.attr.grammar`]); the pattern is an *attribute*.
- In an *alignment-specifier* ([`dcl.align`]); the pattern is the *alignment-specifier* without the ellipsis.
- In a *capture-list* ([`expr.prim.lambda.capture`]); the pattern is the *capture* without the ellipsis.
- In a `sizeof... expression`; the pattern is an *identifier*.
- In a *fold-expression* ([`expr.prim.fold`]); the pattern is the *cast-expression* that contains an unexpanded pack.

§ 4.2. Option 2

NOTE: The idea that a pack-expansion can produce a semicolon-separated "sequence of declarations" is not a problem, but it is novel.

NOTE: We need to permit `friend Ts...;` and `friend typename Ts::Nested...;`, but while keeping `friend int... fs();` and `inline int... xs;` clearly ill-formed. This wording probably achieves the former, but without much regard for the latter. Today, `friend T;` is a *simple-declaration*, consisting of two *decl-specifiers* and no *init-declarator-list*. The second *decl-specifier* is a *defining-type-specifier* which is a *simple-type-specifier*.

NOTE: The identifier in `friend Ts::Nested...;` is in a type-only context because it is a *decl-specifier* of the *decl-specifier-seq* of a *member-declaration*. Therefore Option 2 can omit Option 1's patch to [\[temp.res.general\]](#).

§ 4.2.1. [cpp.predefined]

Add a feature-test macro to the table in [\[cpp.predefined\]](#):

```
__cpp_variable_templates 201304L
__cpp_variadic_friend    YYYYMM
__cpp_variadic_templates 200704L
__cpp_variadic_using     201611L
```

§ 4.2.2. [dcl.type.elab]

Modify [\[dcl.type.elab\]](#) as follows:

4. If an *elaborated-type-specifier* appears with the `friend` specifier as an entire *member-declaration*, the *member-declaration* shall have one of the following forms:

- `friend class-key nested-name-specifieropt identifier ...opt ;`
- `friend class-key simple-template-id ...opt ;`
- `friend class-key nested-name-specifier templateopt simple-template-id ...opt ;`

Any unqualified lookup for the *identifier* (in the first case) does not consider scopes that contain the target scope; no name is bound.

[Note: A using-directive in the target scope is ignored if it refers to a namespace not contained by that scope. [basic.lookup.elab] describes how name lookup proceeds in an elaborated-type-specifier. — end note]

[Note: An *elaborated-type-specifier* can be used to refer to a previously declared *class-name* or *enum-name* even if the name has been hidden by a non-type declaration. — end note]

5. If the *identifier* or *simple-template-id* resolves to a *class-name* or *enum-name*, the *elaborated-type-specifier* introduces it into the declaration the same way a *simple-type-specifier* introduces its *type-name* ([dcl.type.simple]). If the *identifier* or *simple-template-id* resolves to a *typedef-name* ([dcl.typedef], [temp.names]), the *elaborated-type-specifier* is ill-formed.

[Note: This implies that, within a class template with a template *type-parameter* *T*, the declaration `friend class T;` is ill-formed. However, the similar declaration `friend T;` is well-formed ([class.friend]). — end note]

§ 4.2.3. [dcl.type.general]

Modify [dcl.type.general] as follows:

1. The type-specifiers are

type-specifier:

simple-type-specifier ..._{opt}

elaborated-type-specifier ..._{opt}

typename-specifier ..._{opt}

cv-qualifier

type-specifier-seq:

type-specifier *attribute-specifier-seq*_{opt}

type-specifier *type-specifier-seq*

defining-type-specifier:

type-specifier

class-specifier

enum-specifier

[...]

§ 4.2.4. [class.friend]

Modify [class.friend] as follows:

3. A friend declaration that does not declare a function shall have one of the following forms:

```
friend elaborated-type-specifier ...opt ;  
friend simple-type-specifier ...opt ;  
friend typename-specifier ...opt ;
```

[Note: A friend declaration can be the declaration in a *template-declaration* ([temp.pre], [temp.friend]). — end note]

~~If the type specifier in a friend declaration designates a (possibly cv-qualified) class type, that class is declared as a friend; otherwise, the friend declaration is ignored.~~ Each type specifier in such a friend declaration that designates a (possibly cv-qualified) class type declares that class as a friend; each type specifier that does not designate a (possibly cv-qualified) class type is ignored.

[Example 4:

```
class C;  
typedef C Ct;  
struct E;  
  
class X1 {  
    friend C;                // OK, class C is a friend  
};  
  
class X2 {  
    friend Ct;                // OK, class C is a friend  
    friend D;                 // error: D not found  
    friend class D;           // OK, elaborated-type-specifier declares new class  
};  
  
template <class... Ts> class R {  
    friend Ts...;  
};  
  
template <class... Ts, class... Us>  
class R<R<Ts...>, R<Us...>> {  
    friend Ts::Nested...;  
    friend Us...;  
};  
  
R<C> rc;                       // class C is a friend of R<C>  
R<C, E> rce;                   // classes C and E are friends of R<C, E>  
R<int> Ri;                      // OK, "friend int;" is ignored  
  
struct E { struct Nested; };  
  
R<R<E>, R<C, int>> rr;         // E::Nested and C are friends of R<R<E>, R<C, int>>
```

— end example]

§ 4.2.5. [temp.variadic]

Modify [temp.variadic] as follows:

5. A *pack expansion* consists of a *pattern* and an *ellipsis*, the instantiation of which produces zero or more instantiations of the pattern in a list (described below). The form of the pattern depends on the context in which the expansion occurs. Pack expansions can occur in the following contexts:

- In a function parameter pack ([dcl.fct]); the pattern is the *parameter-declaration* without the ellipsis.
- In a *using-declaration* ([namespace.udecl]); the pattern is a *using-declarator*.
- In a friend declaration ([class.mem.general]); the pattern is a *type-specifier*.
- In a template parameter pack that is a pack expansion ([temp.param]):
 - if the template parameter pack is a *parameter-declaration*; the pattern is the *parameter-declaration* without the ellipsis;
 - if the template parameter pack is a *type-parameter*; the pattern is the corresponding *type-parameter* without the ellipsis.
- In an *initializer-list* ([dcl.init]); the pattern is an *initializer-clause*.
- In a *base-specifier-list* ([class.derived]); the pattern is a *base-specifier*.
- In a *mem-initializer-list* ([class.base.init]) for a *mem-initializer* whose *mem-initializer-id* denotes a base class; the pattern is the *mem-initializer*.
- In a *template-argument-list* ([temp.arg]); the pattern is a *template-argument*.
- In an *attribute-list* ([dcl.attr.grammar]); the pattern is an *attribute*.
- In an *alignment-specifier* ([dcl.align]); the pattern is the *alignment-specifier* without the ellipsis.
- In a *capture-list* ([expr.prim.lambda.capture]); the pattern is the *capture* without the ellipsis.
- In a *sizeof... expression*; the pattern is an *identifier*.
- In a *fold-expression* ([expr.prim.fold]); the pattern is the *cast-expression* that contains an unexpanded pack.

[...]

9. The instantiation of a *sizeof... expression* ([expr.sizeof]) produces an integral constant with value N .

10. The instantiation of an *alignment-specifier* with an ellipsis produces $E_1 E_2 \dots E_N$.

11. The instantiation of a *fold-expression* ([expr.prim.fold]) produces:

- $(((E_1 \text{ op } E_2) \text{ op } \dots) \text{ op } E_N)$ for a unary left fold,
- $(E_1 \text{ op } (\dots \text{ op } (E_{N-1} \text{ op } E_N)))$ for a unary right fold,
- $((((E \text{ op } E_1) \text{ op } E_2) \text{ op } \dots) \text{ op } E_N)$ for a binary left fold, and
- $(E_1 \text{ op } (\dots \text{ op } (E_{N-1} \text{ op } (E_N \text{ op } E))))$ for a binary right fold. [...]

x. The instantiation of a friend declaration ([class.friend]) produces a sequence of declarations *friend* E_1 ; *friend* E_2 ; ... *friend* E_N ;

12. The instantiation of any other pack expansion produces a list of elements $E_1, E_2, \dots, E_N$. [Note: The variety of list varies with the context: *expression-list*, *base-specifier-list*, *template-argument-list*, etc. — end note] When N is zero, the instantiation of the expansion produces an empty list.

§ 5. Acknowledgments

I had been sitting on this since posting to the mailing list about it in January 2020. This paper was finally written during C++Now 2023, Feature in a Week. Thus, it is very appropriate to say that it would not exist without that program. Thanks to Jeff Garland, Marshall Clow, Barry Revzin, and JF Bastien for running that program, and all the attendees who helped discuss and review the proposal. Special thanks to Barry Revzin for lots of help with the document itself.

Thanks to Brian Bi and Krystian Stasiowski for further review of the R1 wording.